

# Medii de proiectare și programare

2025-2026

Curs 11

# Conținut

- Clienți web pentru servicii REST
  - JavaScript:
    - Fetch
    - React
- Websocket

- Exemple Promise

# JavaScript -Fetch

- Funcția `fetch()` permite executarea unor apeluri prin rețea asemănătoare cu `XMLHttpRequest` (XHR).
- Diferența: `Fetch API` folosește obiecte `promise`, care permit scrierea unui cod mai simplu și mai clar, evitând callbackurile complexe și API complex al `XMLHttpRequest`.
- Exemplu `XMLHttpRequest`: cererea unui URL, obținerea unui răspuns și parsarea lui ca și JSON.

```
function reqListener() {  
    var data = JSON.parse(this.responseText);  
    console.log(data);  
}
```

```
function reqError(err) {  
    console.log('Eroare:', err);  
}
```

```
var oReq = new XMLHttpRequest();  
oReq.onload = reqListener;  
oReq.onerror = reqError;  
oReq.open('get', './api/some.json', true);  
oReq.send();
```

# JavaScript -Fetch

- Solutia fetch()

```
fetch('./api/slots/1234')
  .then(
    function(response) {
      if (response.status !== 200) {
        console.log('Eroare. Status Code: ' + response.status);
        return;
      }
    }
  )
```

```
    // Examinarea textului din răspuns
    response.json().then(function(data) {
      console.log(data);
    });
  )
  .catch(function(err) {
    console.log('Eroare Fetch ', err);
  });
```

- Rezultatul unui apel *fetch()* este un obiect de tip Stream. Se poate apela metoda *json()*, iar rezultatul va fi un obiect de tip Promise, deoarece citirea streamului se va face asincron.

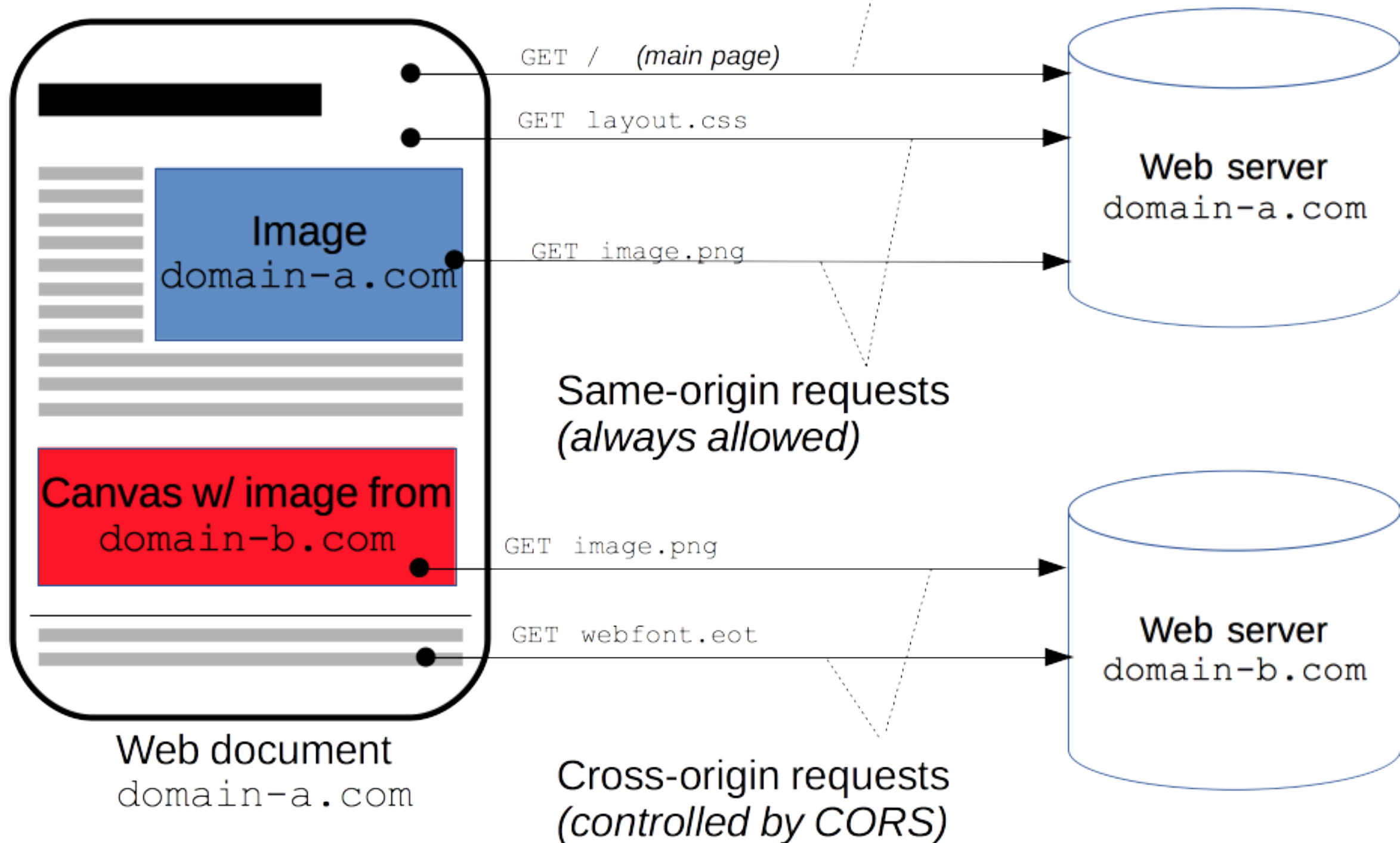
# Fetch Response

- Apelul funcției *fetch* returnează un obiect de tip **response**, dacă obiectul de tip *promise* asociat este îndeplinit.
- Proprietăți utile:
  - **Response.status** - status code asociat răspunsului (întreg, implicit 200).
  - **Response.statusText** - textul asociat status code-ului respectiv (string, implicit "OK").
  - **Response.ok** - permite verificarea rapida daca status-ul este între 200-299 (boolean).
- Alte informații care pot fi obținute (ex. antetele):

```
fetch('http://localhost:8080/users').then(function(response) {  
  console.log(response.headers.get('Content-Type'));  
  console.log(response.headers.get('Date'));  
  
  console.log(response.status);  
  console.log(response.statusText);  
  console.log(response.type);  
  console.log(response.url);  
});
```

# Cross Origin Resource Sharing (CORS)

Main request: defines origin.





# Fetch Response Types

- Când se face o cerere fetch, răspunsul va primi un tip (*response.type*) de tip "*basic*", "*cors*" sau "*opaque*". Aceste tipuri indică de unde provine resursa și poate fi folosit pentru a decide modalitatea de tratare a obiectului de tip response.
  - **basic**: se face o cerere pentru o resursă având aceeași origine ca și cererea. Nu există restricții asupra informațiilor ce pot fi obținute din răspuns.
  - **cors**: se face o cerere pentru o resursă a cărei origine este diferită și care returnează **Cross Origin Resource Sharing (CORS)** în antet. Un răspuns de tip *cors* restricționează antetele care pot fi accesate la `Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified`, și `Pragma`.
  - **opaque**: se face o cerere pentru o resursă având altă origine și care nu returnează CORS în antet. La acest tip de răspuns nu se pot obține datele returnate sau status-ul răspunsului.
    - Nu se poate verifica dacă cererea s-a efectuat cu succes.



# Fetch Response Types

- Se poate defini un mod pentru o cerere fetch, astfel încât doar anumite cereri vor fi apelate:
  - *same-origin*: se execută cu succes doar pentru cereri având aceeași origine, alte cereri vor fi respinse.
  - *cors*: permite cereri având aceeași origine, sau către alte origini care conțin antetele CORs corespunzătoare.
  - *cors-with-forced-preflight*: se verifică anterior că cererea poate fi făcută.
  - *no-cors*: pentru cereri către alte origini care nu au setat antetul CORS, răspunsul va fi de tip opaque.
- Pentru definirea modului se adaugă un parametru cererii fetch:

```
fetch('http://some-site.com/cars/233', {mode: 'cors'})  
  .then(function(response) {  
    return response.text();  
  })  
  .then(function(text) {  
    console.log('Request successful', text);  
  })  
  .catch(function(error) {  
    log('Request failed', error)  
  });
```

# Fetch - înlănțuirea promise

- Pași cerere HTTP:
  - verificarea statusului răspunsului
  - parsarea conținutului pentru a obține obiectul JSON.
- Soluție care necesită doar folosirea informației obținute.

```
function status(response) {  
  if (response.status >= 200 && response.status < 300) {  
    return Promise.resolve(response)  
  } else {  
    return Promise.reject(new Error(response.statusText))  
  }  
}
```

```
function json(response) {  
  return response.json()  
}
```

```
fetch('api/users')  
  .then(status)  
  .then(json)  
  .then(function(data) {  
    console.log('Succes - raspuns JSON:', data);  
  }).catch(function(error) {  
    console.log('Cerere esuata', error);  
  });
```

# Fetch - Verificarea răspunsului

- Un obiect promise *fetch()* va respinge cu un obiect de tip `TypeError`, când apare o eroare de rețea.
- Un apel al funcției *fetch* ar trebui să verifice execuția cu succes a acesteia, verificând că obiectul de tip promise a fost îndeplinit, iar apoi verificând valoarea proprietății `Response.ok`.
- Exemplu:

```
fetch('api/users').then(function(response) {  
  if(response.ok) {  
    return response.json();  
  }  
  throw new Error('Raspunsul nu a fost ok.');
```

```
}).then(function(data) {  
  //folosirea informatiei  
}).catch(function(error) {  
  console.log('Eroare cu functia fetch: ' + error.message);  
});
```

# Fetch - Request

- Funcția *fetch()* poate fi apelată și folosind un parametru de tip Request.
- `Request()` primește aceeași parametrii ca și funcția `fetch()`.

```
var myHeaders = new Headers();
```

```
var myInit = { method: 'GET',  
               headers: myHeaders,  
               mode: 'cors',  
               cache: 'default' };
```

```
var myRequest = new Request('api/users', myInit);
```

```
fetch(myRequest).then(function(response) {  
    return response.json();  
}).then(function(myData) {  
    console.log(myData);  
});
```

- Observație: **Parametrii de tip query trebuie adăugați explicit la cerere (url, antet).**  
Nu există metode speciale care permit transmiterea acestora.

# Antete Fetch

- *Headers* permite crearea/păstrarea/accesarea antetelor unei cereri/unui răspuns.
- Este un dicționar de perechi (nume-antet, valoare-antet):

```
var content = "Hello World";  
var myHeaders = new Headers();  
myHeaders.append("Content-Type", "text/plain");  
myHeaders.append("Content-Length", content.length.toString());  
myHeaders.append("X-Custom-Header", "ProcessThisImmediately");
```

- Se poate transmite și un tablou conținând valorile antetelor:

```
myHeaders = new Headers({  
  "Content-Type": "text/plain",  
  "Content-Length": content.length.toString(),  
  "X-Custom-Header": "ProcessThisImmediately",  
});  
console.log(myHeaders.has("Content-Type")); // true  
console.log(myHeaders.has("Set-Cookie")); // false  
myHeaders.set("Content-Type", "text/html");  
myHeaders.append("X-Custom-Header", "AnotherValue");  
  
console.log(myHeaders.get("Content-Length")); // 11  
console.log(myHeaders.get("X-Custom-Header")); // ["ProcessThisImmediately", "AnotherValue"]  
  
myHeaders.delete("X-Custom-Header");  
console.log(myHeaders.get("X-Custom-Header")); // [ ]
```

# Fetch - Body

- Cererile și răspunsurile pot conține informație/date.
- Informația poate fi de următoarele tipuri:
  - ArrayBuffer
  - ArrayBufferView
  - Blob/File
  - string
  - URLSearchParams
  - FormData
- Sunt definite metode pentru obținerea informației dintr-un răspuns. Toate metodele returnează un obiect de tip promise, care conțin informația.
  - `arrayBuffer()`
  - `blob()`
  - `json()`
  - `text()`
  - `formData()`

# Fetch - Feature detection

- Suportul pentru Fetch API poate fi detectat prin verificarea existenței (în fereastră sau scopul workerului) a obiectelor:
  - Headers
  - Request
  - Response
  - funcția fetch().

Exemplu

```
if (self.fetch) {  
    // execuția cererii fetch  
} else {  
    // folosirea XMLHttpRequest?  
}
```



# React

- React este o bibliotecă JavaScript declarativă, flexibilă și eficientă care permite construirea ușoară și rapidă a interfețelor cu utilizatorul.
- Permite dezvoltatorilor să creeze aplicații web mari, care folosesc date ce se pot modifica în timp, fără a reîncărca toată pagina.
- Obiectivele principale: rapiditate, simplitate, scalabilitate.
- React procesează doar interfețe grafice în aplicații.
  - Corespunde View-ului din șablonul Model-View-Controller (MVC) și poate fi folosit împreună cu alte biblioteci sau frameworkuri JavaScript din MVC (ex. Angular).
- Este dezvoltat și întreținut de o comunitate formată din dezvoltatori de la Facebook, Instagram și dezvoltatori individuali.
- Folosit pentru site-uri precum Netflix, Airbnb, Walmart, etc.
- Mai multe versiuni disponibile
  - React 19 with hooks (latest)



# React - Elemente

- Elementele React sunt cele mai mici construcții ale unei aplicații React. Un element descrie ceea ce trebuie afișat pe ecran.
- Elementele React sunt obiecte simple și ușor de creat. React DOM se ocupă de actualizarea DOM pentru potrivirea cu elementele React.

```
const element = <h1>Hello, world</h1>;
```

- Este numit un nod DOM, pentru că tot ce este conținut de el va fi gestionat de React DOM.
- Aplicațiile dezvoltate folosind React conțin de obicei un singur nod root.
- Pentru redarea unui element React într-un nod root:

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <h1>Hello, world</h1>  
  </React.StrictMode>  
);
```

- Actualizarea unui element: **elementele React sunt *immutable***. Un element odată creat, datele sale nu se mai pot modifica.
- Majoritatea aplicațiilor React apelează *render()* o singură dată.
- Biblioteca React actualizează la afișare doar datele modificate. React DOM compară un element și descendenții săi cu datele anterioare și invocă doar actualizările necesare pentru ca DOM să aibă starea dorită.

# JSX

```
//nu este nici un string, nici HTML.  
const element = <h1>Hello, world!</h1>;
```

- Sintaxa este numită **JSX**, și este o extensie a sintaxei JavaScript.
- Mai strictă decât HTML (<br />).
- Este modul recomandat de descriere în React a interfețelor grafice.
- JSX produce elemente React.
- O componenta poate returna doar un singur element JSX.
- Dacă trebuie returnate mai multe se recomandă folosirea <div>...</div> sau <> ... </>
- Se pot include orice expresii JavaScript în JSX, prin includerea lor între acolade('{ ' ' }')

```
const user = {  
  firstName: 'Popescu',  
  lastName: 'Ana'  
};
```

```
function formatName(user) {  
  return user.firstName + ' ' + user.lastName;  
}
```

```
const element = (  
  <h1>  
    Hello, {formatName(user)}!  
  </h1>  
)  
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    element  
  </React.StrictMode>  
)
```

# Specificarea atributelor folosind JSX

- Se pot folosi ghilimelele pentru specificarea atributelor de tip string:

```
const element = <div tabIndex="0"></div>;
```

- Se folosesc acoladele pentru includerea unei expresii JavaScript ca și valoare a unui atribut:

```
const element = <img src={user.avatarUrl}></img>;
```

- **Nu se pun ghilimele când se includ expresii JavaScript** (expresiile vor fi considerate stringuri)

```
const element = </img>; //!!!
```

- Dacă un tag este gol, se închide cu />, asemănător XML:

```
const element = <img src={user.avatarUrl} />;
```

- Tagurile JSX pot conține fii:

```
const element = (  
  <div>  
    <h1>Salut</h1>  
    <h2>Bine ai venit!</h2>  
  </div>  
)
```

# Componente React

- React are la bază definirea unor **componente React**.
- O componentă poate păstra mai multe instanțe a altei/altor componente (relația *părinte-copil*).
- Componentele permit împărțirea UI în părți independente și reutilizabile, și dezvoltarea independentă a acestora.
- Definirea unei componente (React 18):

```
function Greeting(props){  
  
    return (<h1>Hello, {props.name}</h1>);  
    // sau return (<>Hello, {props.name}</>);  
  
    //gresit  
    //return (Hello, {props.name});  
}
```

- Rezultatul returnat de funcție specifică modul de afișare a componentei.
- **Numele componentelor încep întotdeauna cu litera mare.**
- Exemplu, `<div />` reprezintă un tag DOM, dar `<Greeting />` reprezintă o componentă și este necesar ca *Greeting* să fie disponibil în scop.
- Datele afișate de o componentă React sunt obținute prin proprietăți (*props*) sau din starea componentei (*state*).

# Componente și Props

- Componentele React au două tipuri de date: **starea** și **proprietățile**.
- Când React întâlnește un element reprezentând o componentă definită de dezvoltator, îi transmite atributele JSX ca și un singur obiect (adesea numit “props”), prin parametrii funcției.
- **props** - conține proprietățile definite de componenta care a apelat această componentă.
- **Proprietățile sunt read-only**, o componentă nu trebuie să-și modifice propriile proprietăți:

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <Greeting name="Ana"/> //folosirea componentei  
  </React.StrictMode>  
);
```

```
function Greeting(props) {  
  return (<> {props.name} </>);  
}
```

```
function Greeting({name}) {  
  return (<> {name} </>);  
}
```

```
<button onClick={handleClick}> //fara paranteze când se transmite o funcție!  
  Say hello  
</button>
```

# Compunerea componentelor

- Componentele pot referi alte componente.
- Permite folosirea aceluiași nivel de abstractizare a componentelor: un buton, un form, un dialog, etc sunt toate exprimate ca și componente.
- Exemplu -crearea unei componente care afișează componenta *Greeting* de mai multe ori:

```
function App(){  
  return (  
    <div>  
      <Greeting name="Ana" />  
      <Greeting name="Maria" />  
      <Greeting name="Ion" />  
    </div> );  
}
```

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

# React Component -States

- **Starea** (*eng. state*) unei componente este informația pe care componenta se așteaptă să o gestioneze singură.
- Conține date specifice componentei, care se pot modifica în timp.
- **Starea este definită de dezvoltator.** Dacă informația nu este folosită în **return**, atunci nu face parte din starea componentei.

```
import { useState } from 'react';  
function Counter(){  
  const [count, setCount] = useState(0); //count -variabila de stare, 0-valoarea initiala, setCount-functie  
  function handleClick(){  
    setCount(count+1);  
  }  
  return (<button onClick={handleClick}> Apasat de {count} ori</button>);  
}
```

- Variabila `count` va păstra o parte din starea componentei (pot fi mai multe variabile)
- *Variabilele de stare trebuie considerate immutable*, și nu trebuie modificate în mod direct.
- Orice actualizare a unei variabile de stare a componentei (ex. `count`) se face folosind funcția asociată (ex. `setCount`). De fiecare dată când `setCount()` este apelat, React actualizează starea, determină diferențele dintre starea anterioară și noua stare și injectează o mulțime de modificări DOM-ului corespunzător paginii. În acest fel actualizările UI sunt rapide și eficiente.



# React Component -States

- Observații legate de starea unei componente:

- **NU se modifică starea direct.**

```
const [count, setCount] = useState(0);
```

```
count=count+1; //!eroare
```

**Orice modificare se face prin funcția asociată variabilei** (ex. setCount)

- **O componenta poate avea mai multe variabile de stare, fiecare având funcția ei.**

```
function UserForm(props){
```

```
  const [name, setName] = useState('');
```

```
  const [username, setUsername] = useState('');
```

```
  const [passwd, setPasswd] = useState('');
```

```
  //...  
}
```

- Batch processing pentru stare și actualizarea interfeței

```
function handleClick(){
```

```
  setName(newName);
```

```
  setUsername(newUsername)
```

```
  setPassword(newPassword)
```

```
}
```

//interfața grafică se va actualiza o singura dată



# React Component -Starea

- Datele sunt transmise de la componenta părinte la descendenți.
    - Nici părinții, nici descendenții nu știu dacă o anumită componentă este *stateful* sau *stateless*.
    - Adesea, starea este numită *stare locală* sau *încapsulată*. Starea nu este disponibilă altor componente, doar componentei care a creat-o.
    - O componentă poate alege să transmită starea sa componentelor fii prin intermediul proprietăților:
- ```
function FormattedDate({date}){  
  return (<h2>Data este {date.getDate()}/{date.getMonth()+1}/{date.getFullYear()}</h2>);  
}  
  
function Calendar(){  
  const [date, setDate]=useState(new Date());  
  return (<FormattedDate date={date}/>);  
}
```
- Acest mod de a transmite informația este numit flux de date "top-down" sau "unidirectional". Orice stare este totdeauna definită de o anumită componentă, și orice informație sau UI derivat din acea stare poate afecta doar componentele descendente din arbore.
  - În aplicațiile React, faptul că o componentă este *stateful* sau *stateless* este considerat detaliu de implementare care se poate schimba în timp.
  - Componente *stateless* pot conține componente *stateful* și invers.

# Tratarea evenimentelor

- Asemănătoare cu tratarea evenimentelor corespunzătoare elementelor DOM.
- **Diferențe:**
  - Evenimentele React sunt numite folosind camelCase, nu doar litere mici.
  - Cu JSX se poate transmite o funcție ca și un event handler.

HTML:

```
<button onclick="activateLasers()">
  Activate Lasers
</button>
```

React:

```
<button onClick={activateLasers}> //fara paranteze!
  Activate Lasers
</button>
```

- Nu se poate returna *false* pentru a preveni comportamentul implicit din React. Trebuie apelată explicit metoda *preventDefault*.

HTML: pentru a preveni comportamentul implicit de a deschide o nouă pagină:

```
<a href="#" onclick="console.log('The link was clicked.');" return false">
  Click me
</a>
```

React:

```
function ActionLink() {
  function handleClick(e) {
    e.preventDefault();
    console.log('The link was clicked.');
```

```
  }

  return (
    <a href="#" onClick={handleClick}>
      Click me
    </a>
  );
}
```

# Tratarea evenimentelor

- O altă modalitate de asociere este folosirea sintaxei arrow function:

```
function LoggingButton() {  
  handleClick = () => {  
    //...  
  }  
  //sau handleClick = (e) => { console.log(e)}  
  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}
```

# Input Forms

```
function UserForm(props){

  const [name, setName] = useState('');
  const [username, setUsername] = useState('');
  const [passwd, setPasswd] = useState('');

  function handleSubmit (event){
    let user={id:username,
              name:name,
              passwd:passwd
            }
    //alte operatii
    event.preventDefault();
  }
  return(
    <form onSubmit={handleSubmit}>
      <label>
        Username:
        <input type="text" value={username} onChange={e=>setUsername(e.target.value)} />
      </label><br/>
      <label>
        Name:
        <input type="text" value={name} onChange={e=>setName(e.target.value)} />
      </label><br/>
      <label>
        Passwd:
        <input type="password" value={passwd} onChange={e=>setPasswd(e.target.value)} />
      </label><br/>

      <input type="submit" value="Add user" />
    </form>);
}
```

# Crearea unei aplicații simple React

- **Instalare:** *NodeJs*: <https://nodejs.org/en/>
  - npm: package manager (inclus in nodejs)
- **Creare aplicatie:** `npm create vite@latest my-app -- -- template react`
- Numele directorului cu litere mici!
- Urmati instrucțiunile ...
  - `cd my-app`
- **Pornirea aplicației:**
  - `npm run dev` //va fi disponibilă la <http://localhost:5173/>
  - `npm run build`
- Nu e necesară configurarea folosind babel, webpack/browserify, typescript, etc.

**Important:** NU uitati sa **setați variabila PATH** pentru a include și calea către npm

# Ciclul de viață - Versiuni anterioare

- **Versiuni anterioare** -fiecare componentă avea câteva metode care erau apelate în ciclul său de viață.
- Aceste metode puteau fi redefinite pentru a particulariza componenta.
- Metodele care începeau cu 'will' erau apelate înaintea apariției evenimentului, iar cele cu 'did' erau apelate după apariția evenimentului.
- Metodele apelate când o componentă era creată și adăugată la DOM.
  - *constructor(), componentWillMount(), render(), componentDidMount()*
- *Actualizarea*: cauzată de modificări ale proprietăților sau a stării componente (metodele erau apelate când componenta era re-afișată.)
  - *componentWillReceiveProps(), shouldComponentUpdate(), componentWillUpdate(), render(), componentDidUpdate()*
- *Dezasamblarea*: metoda era apelată când componentă era ștearsă din DOM:
  - *componentWillUnmount()*

# Ciclul de viață - Versiunea curenta

- metoda `useEffects()`

```
import { useState } from 'react';  
import { useEffect } from 'react';
```

```
function UserApp(){  
  const [users, setUsers] = useState([{"passwd":"m","name":"Marinescu Maria","id":"maria"}]);
```

```
  //...  
  useEffect(()=>  
    {  
      //actualizarea interfeței ...  
      getUsers().then(users=>setUsers(users));  
    }, []);
```

Permite executarea unei secvențe de cod după afișarea unei componente și de fiecare data când este actualizata

- `useState`, `useEffect` - **hooks (vers. React 16.8)**

Altele:

- `useContext`
- `useId`
- `useDeferredValue`
- `useTransition`
- `useSyncExternalStore`
- `useInsertionEffect`



# React - arrow functions

```
import React, {useState} from 'react';

const App = () => {
  const greeting = 'React functional component!';
  return <Headline title={greeting} />;
};

const Headline = (props) => {
  const [votes, setVotes] = useState(0);
  const upVote = (event) => {setVotes(votes + 1)};
  return (<div>
    <h1 className="Votes">{props.title}</h1>
    <p>Votes: {votes}</p>
    <p>
      <button onClick={upVote}>Up Vote</button>
    </p>
  </div>);
}
```



# Referințe

- Jake Archibald, *JavaScript Promises: an Introduction*,

<https://developers.google.com/web/fundamentals/getting-started/primers/promises>

- Using Fetch:

[https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)

- Matt Gaunt, *Introduction to fetch()*,

<https://developers.google.com/web/updates/2015/03/introduction-to-fetch>

- Documentație React, <https://react.dev/>
- Documentatie Vite, <https://vite.dev/guide/>
- Versiuni mai vechi React: <https://legacy.reactjs.org/> !!!

WebSockets

# HTTP

## Limitările HTTP:

- *Stateless*:
  - Browserul deschide o conexiune pe portul 80.
  - Trimite o cerere HTTP serverului web.
  - Aplicația server decide ce va face cu cererea, procesează datele, generează răspuns HTML și îl trimite serverului web.
  - Serverul web adaugă antetele HTTP corespunzătoare răspunsului, îl trimite browserului și închide conexiunea.
- Site-urile web trebuie să păstreze informațiile despre utilizatori (ex. cookies).
  - Informația este transmisă între client și serverul web la fiecare cerere.
  - Informațiile adiționale măresc dimensiunea pachetelor transmise și pot face aplicațiile web vulnerabile (securitatea datelor).
- Comunicarea este întotdeauna inițiată de client și fiecare pereche cerere/răspuns este independentă de celelalte cereri/răspunsuri.

# HTTP

- Aplicații *real-time*: burse de valori, vânzarea biletelor, trafic, citirea aparatelor medicale
- Metode:
  - *Polling*, browserul trimite regulat cereri HTTP și primește răspunsul imediat. Este o soluție bună dacă se cunosc intervalele la care mesajele devin disponibile, deoarece se pot sincroniza cererile clienților cu momentele când informațiile sunt disponibile pe server.
  - *Long-polling*, browserul trimite o cerere la server și serverul păstrează cererea deschisă pentru o perioadă prestabilită de timp. Dacă în acea perioadă se primește o notificare, serverul trimite clientului un răspuns care conține și notificarea. Dacă nu se primește nici o notificare, serverul trimite un răspuns pentru a încheia cererea clientului.
  - *Streaming*, browserul trimite o cerere completă dar serverul trimite și păstrează un răspuns incomplet ce este actualizat continuu și păstrat deschis pe termen nelimitat (sau o perioadă prestabilită de timp). Răspunsul este actualizat de fiecare dată când trebuie trimisă o notificare, dar serverul nu semnalează terminarea răspunsului, păstrând astfel conexiunea deschisă pentru mesaje ulterioare. Pot să apară probleme din cauza păstrării într-un buffer a răspunsurilor.

# WebSockets

- WebSocket-urile sunt o **conexiune persistentă, bi-direcțională, full-duplex** de la un browser web la un server.
- După ce conexiunea a fost stabilită, ea rămâne deschisă până când clientul sau serverul decide să o închidă.
- Cât timp conexiunea este deschisă, clientul și serverul își pot trimite mesaje unul altuia în orice moment de timp.
- Programarea web devine astfel bazată pe eveniment, și nu doar inițiată de utilizator. Este *stateful*.
- O singură aplicație server care rulează știe de toate conexiunile, permițând comunicarea cu orice număr de conexiuni deschise în orice moment de timp.

# Protocolul WebSocket

- În 2011, IETF a standardizat protocolul WebSocket sub denumirea de RFC 6455.
- De atunci, majoritatea browserelor Web implementează cod API pe partea de client care suportă protocolul WebSocket.
- S-au dezvoltat biblioteci în Java, .NET, Ruby, Objective C, JavaScript, etc. care implementează protocolul WebSocket.
- Browsere care suportă WebSockets: nativ în Chrome, Firefox, Opera și Safari (inclusiv Safari pentru dispozitive mobile), Internet Explorer.
- Orice browser care nu suportă WebSockets poate folosi soluția *polyfill Flash*.

# Protocolul WebSocket - Handshake

Pentru stabilirea unei conexiuni WebSocket, **clientul trimite o cerere HTTP pentru un *handshake* WebSocket:**



```
GET /echo HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: http://example.com
```



# Protocolul WebSocket - Handshake

- Dacă serverul acceptă cererea de actualizare a protocolului, va returna un răspuns **HTTP 101 Switching Protocols**:



**HTTP/1.1 101 Switching Protocols**

**Upgrade: websocket**

**Connection: Upgrade**

**Sec-WebSocket-Accept: HSmrc0sMlYUkAGmm5OPpG2HaGWk=**

**Sec-WebSocket-Protocol: echo**

- După ce serverul returnează răspunsul 101, **protocolul la nivel de aplicație se va schimba din HTTP în WebSockets**, care va folosi conexiunea TCP stabilită anterior. Din acest moment, **ambii participanți pot trimite/primi mesaje în orice moment de timp**.





# WebSocket - URI

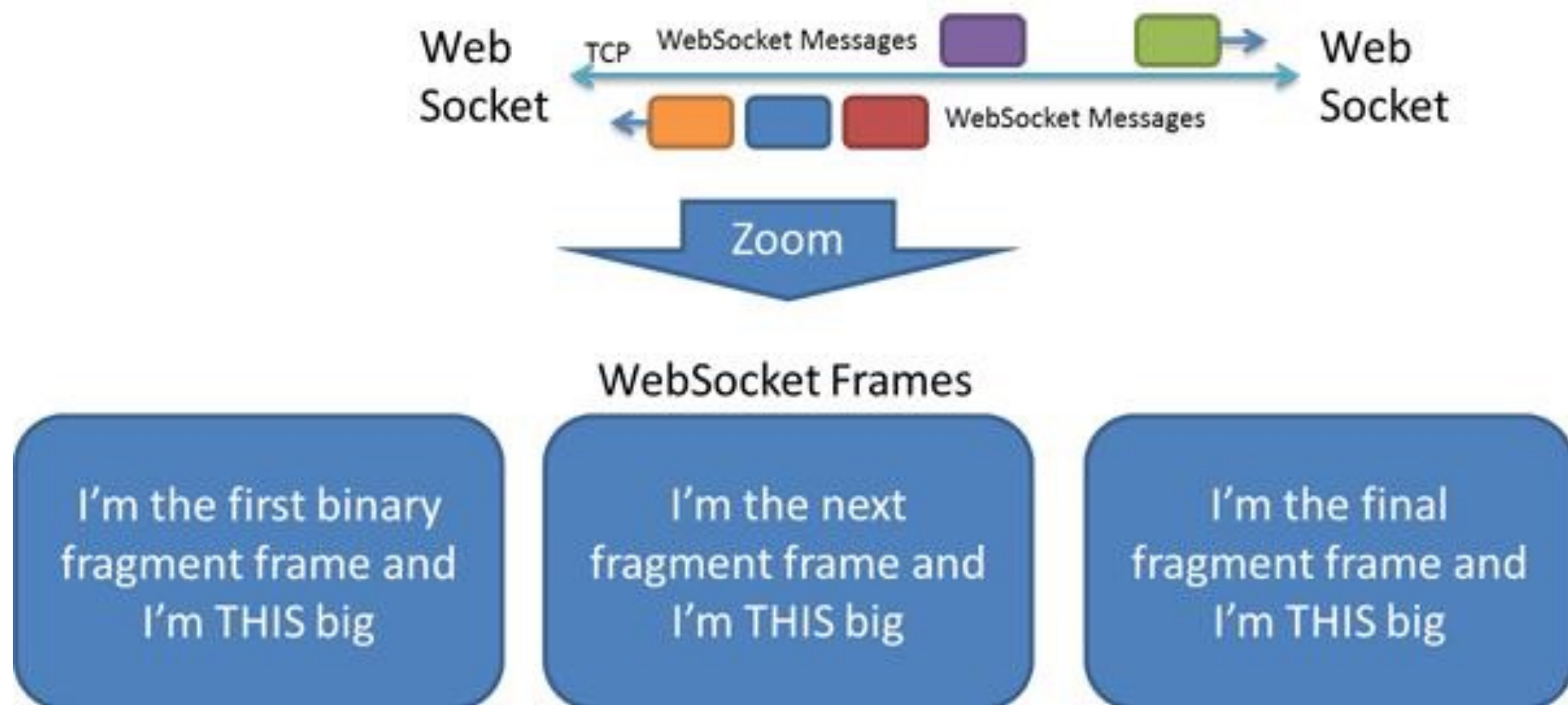
- Protocolul WebSocket definește două formate URI noi, similare cu formatele HTTP:
- `"ws:" "://" host [ ":" port ] path [ "?" query ]` modelată după formatul "http:"
  - Portul implicit este 80.
  - Este folosit pentru conexiuni nesecurizate (necriptate).
- `"wss:" "://" host [ ":" port ] path [ "?" query ]` modelată după formatul "https:"
  - Portul implicit este 443.
  - Este folosit pentru conexiuni securizate peste Transport Layer Security (TLS).

`ws://example.com`

`ws://example.com:8080/echo`

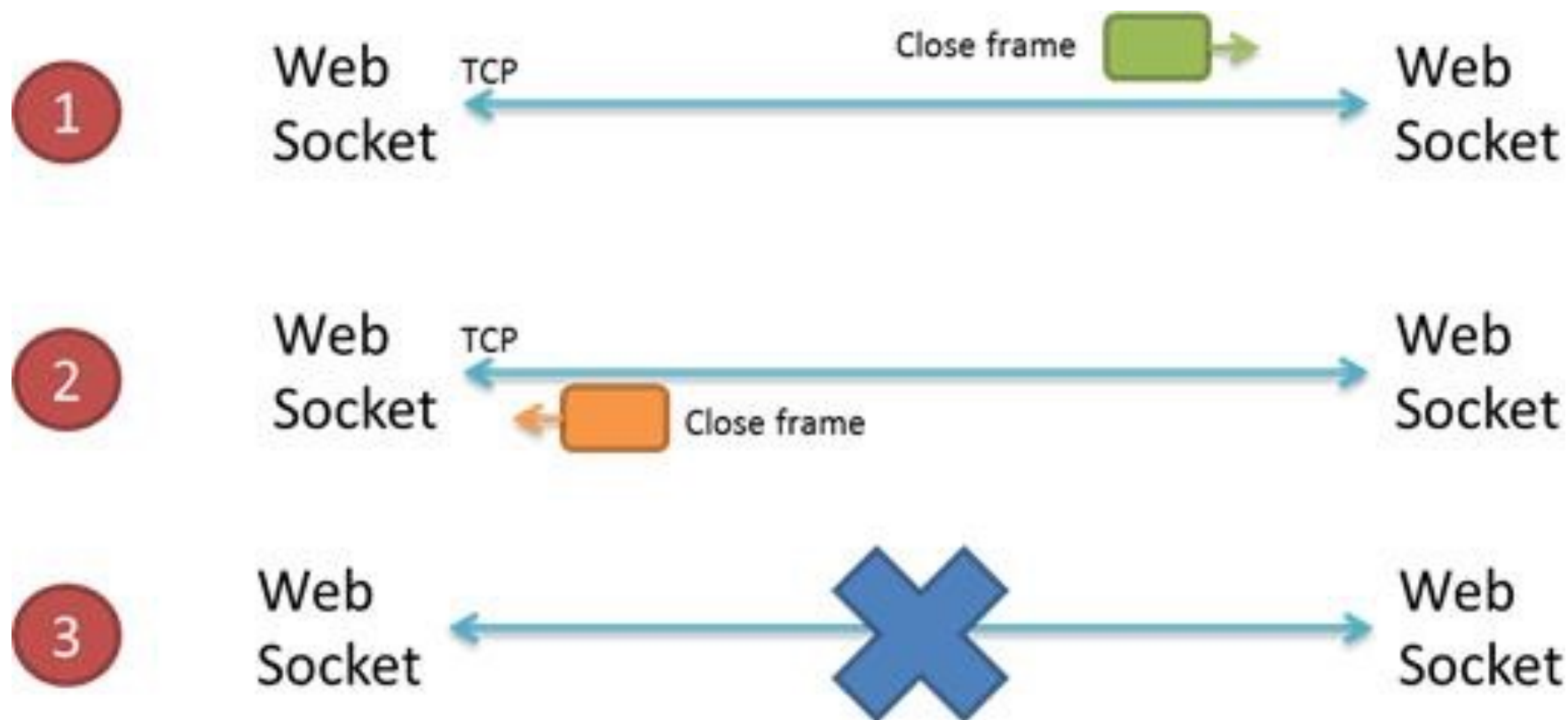
# Mesaje WebSocket

- După o înțelegere (handshake) cu succes, aplicația și serverul web pot interschimba mesaje WebSocket.
- Un mesaj este compus dintr-o secvență de unul sau mai multe fragmente de mesaj/ date numite “frames”. Fiecare frame conține informații cum ar fi:
  - Dimensiunea/lungimea framelui.
  - Primul frame din mesaj va conține tipul mesajului (text sau binar).
  - Un flag (*FIN*) care indică dacă acesta este ultimul frame din mesaj.



# Închiderea unui WebSocket

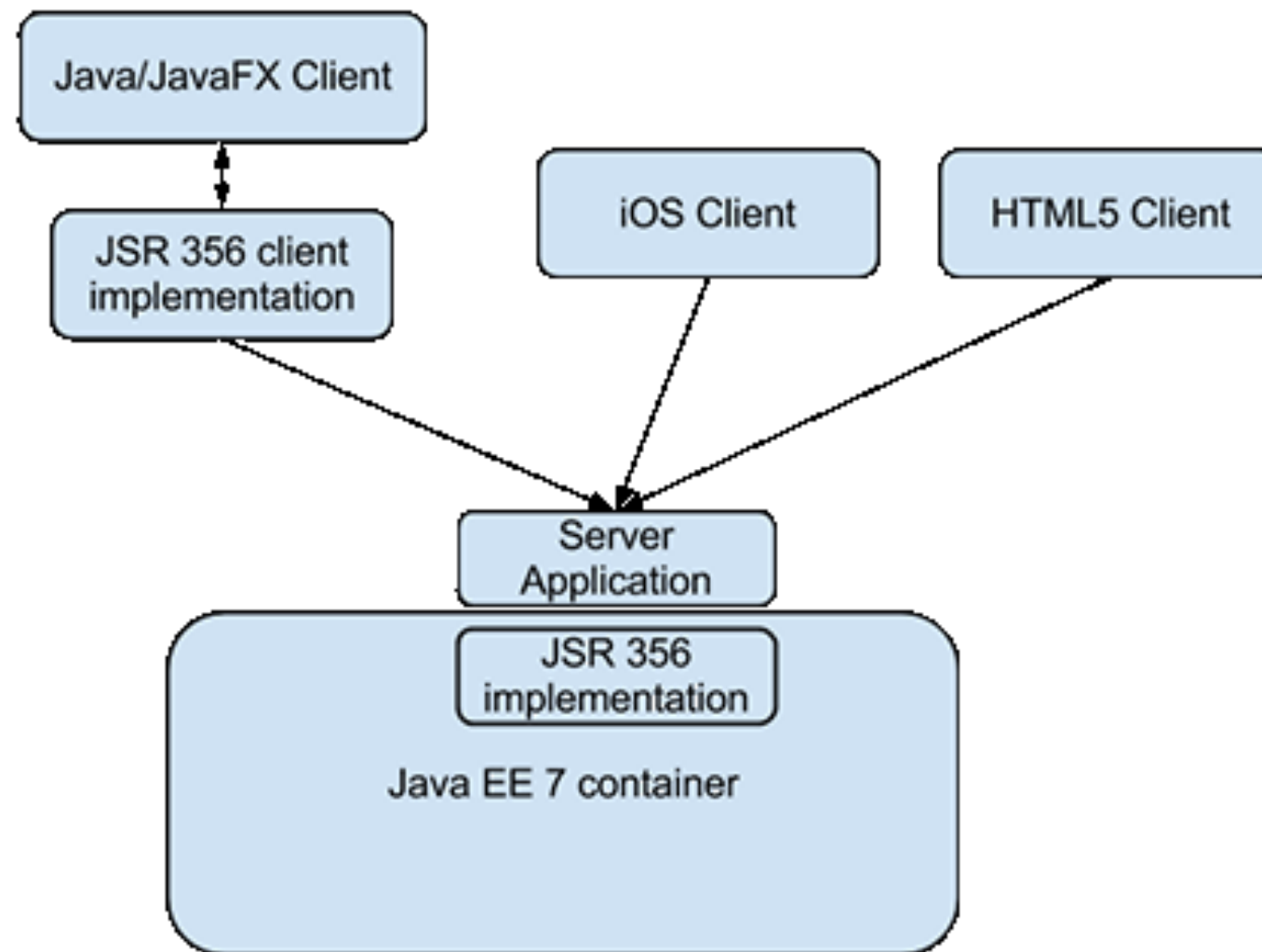
- Fiecare participant la conexiune (clientul sau serverul) poate iniția o cerere de terminare a înțelegerii (*handshake*).
- Un frame special – un frame *close* – este trimis celuilalt participant. Frame-ul *close* poate să conțină opțional un motiv pentru terminare și un status code.
- Protocolul definește un set de valori ce pot fi folosite pentru status code.
- Cel care trimite frame-ul *close*, nu trebuie să mai trimită alte mesaje după trimiterea acestuia.
- Când celălalt participant primește un frame *close*, el va răspunde cu propriul frame *close*. Poate să trimită mesajele netrimise anterior (din buffer) înainte de a trimite frameul *close*.



# WebSockets API - Java

- **JSR 356**, Java API for WebSocket, [specifică API-ul care poate fi folosit de dezvoltatorii Java când vor să integreze/folosească WebSocket-uri în aplicațiile lor, atât pe partea de server cât și pe partea de client Java.](#)
- Fiecare implementare a protocolului WebSocket care pretinde să respecte JSR 356 trebuie să implementeze acest API.
- Dezvoltatorii pot dezvolta aplicațiile lor bazate pe WebSocket independent de implementarea WebSocket.
- JSR 356 face parte din standardul Java EE 7.
- Toate serverele de aplicații care respectă Java EE 7 vor avea/conține o implementare a protocolului WebSocket care respectă standardul JSR 356.

# WebSockets API - Java



- Un client Java poate folosi o bibliotecă client care respectă JSR 356 pentru tratarea problemelor (eng. *issues*) specifice protocolului WebSocket.
- Alți clienți (iOS, HTML5) pot folosi alte implementări (care nu sunt bazate pe Java) dar care respectă specificația RFC 6455 pentru a comunica cu aplicația server.

# WebSockets API - Java

- JSR 356 folosește adnotări și injectare.
- Sunt suportate **două modele de programare**:
  - *Bazate pe adnotări*. Se folosesc clase POJO adnotate, cu ajutorul cărora dezvoltatorii pot trata evenimentele ce apar în ciclul de viață al unui WebSocket.
  - *Bazate pe interfețe*. Dezvoltatorii pot trata evenimentele prin implementarea interfeței *Endpoint* care conține metodele corespunzătoare evenimentelor ce pot să apară.
- **Evenimentele din ciclul de viață**:
  - Un client inițiază conexiunea trimițând o cerere de tip HTTP handshake.
  - Serverul răspunde cu un handshake.
  - Conexiunea este stabilită. Din acest moment **conexiunea este simetrică, bi-direcțională**.
  - Ambii participanți (clientul și serverul) **pot să trimită și să primească mesaje**.
  - Unul dintre participanți **închide conexiunea**.
- Majoritatea evenimentelor din ciclul de viață pot fi mapate la metode Java, indiferent de abordarea folosită (adnotări sau interfețe).

# Java WebSockets API - Anotări

- Un endpoint care acceptă cereri WebSocket (serverul) poate fi orice POJO (Plain Old Java Object) annotat cu `@ServerEndpoint`.
- Această anotare specifică containerului (web) că clasa annotată ar trebui considerată un endpoint WebSocket.
- Elementul anotării specifică calea către endpoint-ul WebSocket.

```
@ServerEndpoint("/hello")
```

```
public class MyEndpoint { }
```

```
@ServerEndpoint("/hello/{userid}")
```

```
public class MyEndpoint { }
```

- valoarea variabilei `{userid}` poate fi obținută în metodele corespunzătoare ciclului de viață folosind anotarea `@PathParam`.



# Java WebSockets API - Adnotări

- Un endpoint care inițiază o conexiune WebSocket (clientul) poate fi un POJO adnotat cu `@ClientEndpoint`.
- `ClientEndpoint` nu accepta variabile de cale (ex. {userid}) deoarece nu așteaptă cereri.

`@ClientEndpoint`

```
public class MyClientEndpoint {}
```

- Inițierea unei conexiuni WebSocket, folosind abordarea bazată pe adnotări:

```
javax.websocket.WebSocketContainer container =  
javax.websocket.ContainerProvider.getWebSocketContainer();
```

```
container.connectToServer(MyClientEndpoint.class,  
new URI("ws://localhost:8080/hello"));
```

- Clasele adnotate cu `@ServerEndpoint` sau `@ClientEndpoint` sunt numite endpoint-uri adnotate.



# Java WebSockets API - Adnotări

- După ce a fost stabilită o conexiune WebSocket se creează un obiect de tip `Session` și metoda adnotată cu `@OnOpen` va fi apelată.
- Metoda poate avea diferiți parametri:
  - Un parametru de tip `javax.websocket.Session` specificând sesiunea (obiectul Session) creată.
  - Un obiect de tip `EndpointConfig` care conține informații despre configurația endpoint-ului.
  - Zero sau mai mulți parametri de tip String adnotați cu `@PathParam`, care referă variabilele de cale din calea endpoint-ului(server).

## `@OnOpen`

```
public void myOnOpen (Session session) {  
    System.out.println ("WebSocket opened: "+session.getId());  
}
```